PlacedOn — Master Product & AI Plan

PlacedOn · CTO / Eng / AI

July 15, 2026

CONTENTS

PlacedOn — Master Product & AI Plan

Prepared as: CTO · Lead Engineer · Head of AI/ML
Date: 2026-07-15 · **Live:** https://placedon.comSMARTPANTS_PRESERVED_393

This document combines the CTO-level audit + product decisions with the deep engagement, voice, and AI/ML working-prototype plan.

Book I — Audit & Product Decisions

PlacedOn — CTO Audit & Product / AI Plan

Author's stance: written as if I own three hats — CTO, lead engineer, and head of AI/ML. **Date:** 2026-07-15 · **Live:** https://placedon.comSMARTPANTS_PRESERVED_410 · **Scope:** what exists, what's fake, what to build, and how the AI should actually work.

Companion docs: `PLACEDON_LIVE_INTERVIEW_HR_COPILOT_PLAN.md` (the 8-slice V1 spec), `IMPLEMENTATION_LOG.md` (build state). This doc is the *audit + decisions + AI roadmap* layer on top of that.

Part A — What is actually live today (grounded in code)

Area	Marketing claim on site	Reality in code	Verdict
Interview	"Voice or text, adaptive"	Real FastAPI engine exists (`pipeline/`, `llm/`, `websocket_router`), but the live frontend has `isLiveBackend()==false` → **scripted mock** (`api.ts`). Feels one-way because it *is* a canned demo.	⚠️ Prototype, disconnected
Voice	"Speak your answers"	**No mic/recorder/STT in the frontend at all.** Backend TTS = macOS `say`; STT path unwired.	❌ Not a feature
Candidate dashboard / matches / applications	Live profile, matches, applications	Labeled "Preview state" / "Sample pipeline · illustrative" → **mock data**, empty shells.	❌ Mock
Report card / passport	"Candidate-controlled evidence"	`TrustPassport.tsx` shows an **"Overall skill score"** (`report.overall`) — a universal score the plan explicitly bans. Sample evidence only.	❌ Wrong + mock
Employer / job posting	"Post a role, find candidates"	**Slice 1 built by us**: DB + RLS + `/v1/jobs`, `/role-dna`, `/reality-card` (real, authenticated). No UI yet.	🟡 Backend only
HR Copilot	"Search candidates in plain English"	Does not exist.	❌ Not built
Introductions / consent gate	"Candidate approves contact"	Does not exist.	❌ Not built
Matching algorithm	"Explained role fit"	Does not exist (mock cards only).	❌ Not built

Three trust-integrity defects to fix before anything else (P0):

1. **Remove the universal "Overall skill score"** from `TrustPassport.tsx`. It contradicts the entire product thesis (evidence-per-role, no employability score) and is a legal/fairness liability (NYC LL144, EU AI Act treat a single hiring score as high-risk).
2. **Stop the silent demo fallback.** When there's no live backend, the interview must say "This is a scripted preview," not impersonate a real adaptive interview. Silent mock = the "one-way, fake" feeling you flagged.
3. **De-mock the dashboards or clearly gate them** behind "coming soon" so the live site stops advertising features that don't persist anything.

Part B — The product flows that are missing (the "how does it actually work")

Each answers one of your questions with a concrete flow + example.

B1. WHERE DOES A CANDIDATE GO, APPLY, AND WHERE IS DATA STORED?

```
Sign up (Supabase Auth) → profiles row (role=candidate)
 → /candidate: one clear next action (state machine)
 → consent + preferences (candidate_preferences)
 → pick a role direction (junior backend for pilot)
 → adaptive text interview (interview_sessions + interview_turns, answer_encrypted)
 → async report card built (report_cards + report_card_items + trait_evidence)
 → candidate REVIEWS before anyone sees it (accurate / add-context / dispute / hide)
 → approves an Evidence Passport (visibility scope)
 → sees explained matches (matches) → expresses interest
 → employer requests intro (intro_requests) → candidate accepts → contact released
```

Storage: all in Supabase Postgres with **Row Level Security** as the source of truth (candidate reads only their own rows; employers read only role-scoped, candidate-approved evidence). Raw transcripts (`answer_encrypted`) are **never** returned to employers.

Example: Aditi signs up → consents → picks "junior backend" → 12-min text interview → gets a private report ("Traces assumptions before proposing a fix — supported; no code-review example yet — needs evidence") → hides one item, adds context to another → approves passport for matching. Her raw answers never leave her account.

B2. FIXING THE "ONE-WAY, SCRIPTED" INTERVIEW (MAKE IT FEEL LIKE A CONVERSATION)

The engine is adaptive; the *experience* isn't wired. Fixes:

- **Connect the real runtime** (`WS /v1/interviews/{id}/live`), retire the mock path.
- Each AI question must **reference the candidate's actual last answer** ("You mentioned normalizing the address first — what would you check next if that passed?"). The `pipeline/` already extracts observations; surface that continuity in the prompt.
- **Acknowledge every answer** ("Saved") + a calm "thinking" status while the next question is generated — no fake typewriter, no dead air.
- **Stop on evidence sufficiency**, not a fixed count (already in the plan) — so it doesn't feel like a form.
- Let the candidate **ask for clarification** or say "can you rephrase" — a two-way affordance.

B3. HOW THE REPORT CARD WORKS (AND SALARY / STRENGTHS / GROWTH)

Not a verdict — a **candidate-controlled, per-role evidence summary**. Structure:

- **Evidence-supported strengths** (claim + exact quote + role relevance + confidence band `supported / emerging / needs_more_evidence`). **No numeric score.**
- **Growth conditions / missing evidence** — phrased as "no cross-team example was observed," never "weak communication."
- **Role compatibility** — which Role DNA signals are supported vs not-yet-evidenced.
- **Work-environment alignment** — candidate preferences vs the employer's **Job Reality Card** (which carries the **salary range** where allowed — so "what salary can I expect" is answered by the *employer's disclosed range*, not an AI guess).
- **Salary/advantages/disadvantages** are reframed: *advantages* = "strengths supported by evidence for this role"; *disadvantages* = "open questions / conditions to explore" (e.g., "on-site schedule differs from your stated preference"). Never a global "good/bad."

B4. MATCHING: WHICH JOB FITS, WHICH ROLE, WHERE TO APPLY

A match is **a list of independent, explained dimensions** — not one hidden score:

Dimension	Source	Candidate sees	Employer sees
Role evidence	approved items vs Role DNA	"3 of 4 signals supported"	supported / emerging / missing
Job constraints	candidate prefs vs Reality Card	"hybrid Bengaluru ✓, salary in range ✓"	relevant approved prefs only
Growth	role gaps vs learning prefs	"you'd build code-review evidence here"	suggested human follow-up
Availability	candidate state	"explore now?"	can request intro?

Retrieval may use a bounded internal relevance score, but the candidate/employer **always** see the dimension breakdown, never a bare number. "Where should I apply" = the ranked, explained queue on `/candidate/matches`.

B5. HOW HR FINDS A CANDIDATE (PROMPT → RESULTS)

```
HR types plain English → LLM parses to TYPED criteria JSON
 → deterministic policy filter (reject protected/proxy/auto-decision)
 → retrieve ONLY role-scoped, candidate-approved evidence (RLS)
 → structured filter + semantic (embedding) retrieval
 → rank by role relevance / evidence strength / consent / prefs
 → templated explanation with EVIDENCE CITATIONS + missing signals
 → HR saves / passes / requests intro (all logged to audit_events)
```

Example prompt: "Junior backend candidates who showed structured debugging, open to Bengaluru hybrid, prefer regular feedback." → parsed chips shown, each result cites the exact approved quote, states what's missing, suggests a human follow-up. Unsafe prompts ("young men from top colleges") are **refused or rewritten**.

B6. HOW A COMPANY POSTS A JOB (BUILT AT THE DB/API LAYER ALREADY)

`/employer/jobs/new` → **Role DNA Builder** (business problem, first-90-day outcome, 3–5 success signals, must-haves vs nice-to-haves, human follow-up) + **Job Reality Card** (work mode, location, **salary range**, team, process, response SLA). A job **cannot enter search** until complete (enforced server-side by `compute_search_ready`). This is Slice 1 — backend done, UI next.

B7. HOW THE AI GUIDES THE CANDIDATE AFTER THE INTERVIEW

- "Here are the 3 signals your evidence supports, and 2 that need a human conversation."
- "These 4 roles are worth exploring because … ; here's the salary range each disclosed."
- "To strengthen readiness for X, add a project showing code-review collaboration, or answer one follow-up." Guidance is **actionable and honest**, never "you will get hired at Company Y."

Part C — The candidate profile decision (simple vs social) — my call

Recommendation: evidence-first passport, NOT a social network. Do not build feeds, follows, or open DMs in V1.

Why (CTO reasoning):

- **The moat is trust + evidence, not a social graph.** LinkedIn owns the social graph; competing there is a losing, expensive fight. Our defensible asset is a *candidate-owned, consent-controlled Evidence & Outcome graph* (hard to copy because it needs repeated trust behavior).
- **Social features import huge liabilities**: moderation, harassment, spam, fake accounts, addictive engagement metrics that conflict with a *calm, high-trust* hiring product. It also invites bias (popularity ≠ competence).
- **It dilutes the promise.** The candidate's win is "I'm represented fairly and I control who sees what," not "I got 40 likes."

What the profile *should* be:

- A **private-by-default Evidence Passport**: per-role supported strengths (with quotes), growth conditions, preferences, and visibility controls.
- A **shareable link** the candidate can turn on (a credible, evidence-backed page) — light "presence," not a feed.

- **Chat only inside the consented introduction flow** (candidate ↔ employer *after* mutual interest), never open social messaging.
- Later, optional: candidate can attach **work samples** and re-interview to strengthen evidence. That's "engagement" that compounds trust, not vanity.

If you later want community (peer prep, mentorship), run it as a *separate, clearly-bounded* surface — never mixed into the assessment/evidence path.

Part D — AI / ML / "neural network" roadmap (head-of-AI honest take)

You're excited to "train a neural network." As head of AI/ML, my job is to channel that into something that **works, is fair, and won't get us sued** — not a black-box hiring brain.

D0. THE HARD RULE

Do NOT train a proprietary model that outputs hiring decisions or a person-score in V1. Reasons: no data yet, severe bias risk, and hiring AI is legally *high-risk* (NYC LL144 bias audits, EU AI Act, ILO warnings). A model that decides "hire/don't" is the fastest way to a lawsuit and an unfair product. The AI **informs** a human; it never decides.

D1. WHAT THE "AI" ACTUALLY IS IN V1 (THIS IS REAL ML, DONE RIGHT)

1. **Foundation LLMs as constrained components** (Claude). Already prototyped in `llm/` (`generator`, `judge`, `claude_axis`). Rules: strict JSON schemas, low temperature, a **judge** that checks every extracted claim against the source answer, and **deterministic code owns policy/authorization** (never the model).
2. **Sentence embeddings for retrieval** — you already depend on `sentence-transformers`. Use it (or an embedding API) to power **semantic evidence search** for HR Copilot and matching. Store vectors in **pgvector** on Supabase. This is a legitimate neural-net use: an *encoder for retrieval*, not a decision-maker.
3. **Rubric-based, cited, calibrated scoring** — per **Role DNA signal**, with confidence bands, always tied to evidence IDs. Not a universal score.

D2. THE STAGED ML ROADMAP (HOW WE ACTUALLY GET "SMART")

```
Stage 0 (now): LLM extract → judge → confidence band → candidate review.  Ship this.
Stage 1: Embeddings + pgvector semantic retrieval for HR Copilot + matching.
Transparent.
Stage 2: Build the EVALUATION SET — consented, de-identified interview cases with
         expected permitted observations, forbidden observations, evidence IDs,
         and HR-prompt→policy expectations. This is the real asset.
Stage 3: Calibrate the LLM-judge against the eval set (groundedness, fidelity,
         harmful-inference rate). Measure, don't vibe.
Stage 4: ONLY after outcome data (30/90-day check-ins) accumulates → train a
         LEARNED RE-RANKER (gradient-boosted trees or a small MLP) that reorders
         candidate↔role relevance. Features = embeddings + structured Role-DNA
         match + preference alignment. It RE-RANKS for a human; it never rejects.
         Ship it behind a fairness gate (adverse-impact ratio, subgroup metrics).
```

D3. HOW WE'D TRAIN THE FIRST MODEL WORTH TRAINING (THE RE-RANKER)

- **Data:** consented, de-identified `(role dna, approved evidence, preferences, job reality) → outcome` rows from `outcome_checkins`. Start with hundreds, not millions — this is small-data, so **gradient-boosted trees (LightGBM) or logistic ranking**, not a deep net. Deep nets need data we won't have and hurt explainability.
- **Features:** evidence-embedding similarity to Role DNA signals, count of supported signals, preference/reality alignment flags, recency. **Explicitly exclude** any protected or proxy feature (college prestige, name, location-as-identity, gaps).
- **Label:** did the intro lead to a useful human outcome (advanced / offer / mutual "worth it")? Never "is this person good."
- **Evaluation:** offline ranking metrics (NDCG) **plus fairness** — adverse-impact ratio and subgroup calibration on any available demographic *audit* set (held separately, never used as a feature).
- **Guardrail:** output is a *bounded relevance bucket* feeding the explained match UI. A red team + bias audit must pass before it's live. If fairness fails, we don't ship it — we keep the transparent retrieval.

D4. CANDIDATE "PROFILE BUILDING" ML (WHAT POWERS THE PASSPORT)

Pipeline (mostly Stage 0, already prototyped in `pipeline/`):

```
answer → extract role-relevant observations (LLM, schema'd)
       → judge fidelity vs the exact answer text (reject hallucinated claims)
       → assign confidence band + role relevance
       → candidate review (accurate/context/dispute/hide) = human-in-the-loop label
       → approved items become the passport + retrieval corpus
```

The candidate's disputes/edits are **gold training signal** for Stage 3 calibration — the trust loop *is* the data flywheel.

D5. VOICE (REAL, NOT MACOS `SAY`)

- **V1 = text-first** (the plan is right: text is more inclusive, auditable, translatable, and **avoids judging accent/fluency**, which is a fairness landmine).
- **Voice as opt-in fast-follow:** browser `MediaRecorder` /Web Audio → stream over WSS → **production STT** (Deepgram / AssemblyAI / faster-whisper on a GPU worker) → the *same text pipeline*. TTS via **ElevenLabs / Azure / OpenAI**, replacing macOS `say`. The backend already has the `interaction_layer.voice` seam to plug a real provider into.
- **Fairness guardrail:** voice is transcribed to text and assessed **as text only**. We never score tone, accent, filler words, or speed. Store consent + provider + fallback-to-text.

Part E — Prioritized roadmap (what to build, in order)

P0 — Trust integrity (small, do first):

1. Remove the universal "Overall skill score" from `TrustPassport.tsx`; replace with per-signal confidence bands.
2. Make the interview honestly label scripted-preview vs live.
3. Gate/label mock dashboards.

Then the V1 slices (Slice 1 backend done):

- **Slice 1 (finish):** employer Role DNA + Reality Card **UI** + integration tests. *(in progress)*
- **Slice 2:** candidate consent + preferences + interview creation (real identity + storage).
- **Slice 3:** resilient live text interview wired to the real runtime (retire the mock).
- **Slice 4:** candidate Evidence Report Card (review/hide/dispute, no universal score).
- **Slice 5:** explained matching (dimensions, salary from Reality Card).
- **Slice 6:** HR Copilot (prompt → policy → RLS retrieval → citations).
- **Slice 7:** consent-gated introductions + the only chat surface.
- **Slice 8:** PMF instrumentation + outcome check-ins (feeds the eval set + re-ranker).
- **Fast-follow:** pgvector semantic retrieval (Stage 1), then voice (opt-in), then the re-ranker (Stage 4, gated).

New features worth adding (post-V1, ranked):

1. Work-sample attachments (compounds evidence trust).
2. Re-interview to strengthen a specific signal.
3. Candidate "readiness" guidance (what evidence to build, honestly).
4. Employer calibration workspace (consistent Role DNA across managers).
5. Shareable passport link.
6. WhatsApp/Hinglish candidate support (India wedge) — *after* the text model is trusted.

Part F — Backend module map (so we build on it, not around it)

`Code/PlacedOn/backend/app/`:

- `main.py` — FastAPI app, CORS, router mounts. · `config.py` — settings.
- `websocket_router.py` + `live_runtime.py` + `session_manager.py` (Redis) — live interview transport.
- `pipeline/` — `jd_parser`, `planner`, `question_strategy`, `conversation_orchestrator`, `context_builder` — the adaptive engine (reusable for Slice 3).
- `llm/` — `generator`, `judge`, `claude_axis`, provider clients — the constrained-LLM layer.
- `state_compressor.py`, `trust_trigger.py` — state contraction + anomaly signals.
- `interaction_router.py` + `interaction_layer.voice` — **experimental** voice seam (STT factory, audio frames). Replace TTS (`tts_service.py` macOS `say`) + wire real STT here for the voice fast-follow.
- `api_routes.py` (CSV ingest / rating / export) + `analytics_router.py` — **eval/data pipeline**, not product API.
- `waitlist_router.py` — leads.
- **`deps.py` + `jobs.py` + `v1_router.py`** — our new authenticated `/v1` product API (Slice 1).

Target data model (Supabase): built = `profiles`, `demo_requests`, `companies`, `organization_members`, `jobs`, `role_dna_signals`, `job_reality_cards`. To build = `candidate_preferences`, `interview_sessions`, `interview_turns`, `report_cards`, `report_card_items`, `trait_evidence`, `trait_reviews`, `matches`, `hr_search_sessions`, `hr_search_results`, `employer_candidate_actions`, `intro_requests`, `outcome_checkins`, `audit_events`, `product_events`, `model_evaluations` — each with RLS. (Full contract in `PLACEDON_LIVE_INTERVIEW_HR_COPILOT_PLAN.md` §10.)

The one-line test for every feature

> Does this help the candidate feel accurately represented and in control, while helping the employer see role-relevant evidence and make a better *human* decision?

If not clearly yes, it doesn't go in V1.

Book II — Engagement, Voice & AI Working-Prototype Plan

PlacedOn — Engagement + Voice + AI Working-Prototype Plan

Role: CTO + head of AI/ML · **Date:** 2026-07-15 · **Status:** approved-for-build plan (working prototype, not a throwaway V1) **Source of truth:** `docs/CTO-AUDIT-AND-PRODUCT-PLAN.md` (audit + decisions), `IMPLEMENTATION_LOG.md` (build state). **Note:** `PLACEDON_LIVE_INTERVIEW_HR_COPILOT_PLAN.md` is referenced by both docs but is **missing from this repo** — the audit doc's summaries (slices, §10 data contract, §11 policy gates) are treated as its canonical summary. Restore or re-export the file; until then this plan is self-contained.

Non-negotiable invariants (repeated because every section must respect them):

1. No universal person-score. Ever. Evidence is per-role-signal with confidence bands.
2. Voice answers are transcribed and assessed **as text only** — never accent, tone, fluency, filler words, or speed.
3. Supabase RLS is the source of truth for tenant/consent scope. The product API uses the anon key + user JWT (`app/deps.py` already does this correctly — extend, don't bypass).
4. A human makes the hiring decision. AI extracts, retrieves, ranks, explains — it never rejects or decides.
5. No open social feed, followers, likes, or public DMs. The only messaging is the consented intro chat.
6. Deterministic code owns policy/authorization. The LLM never gates access, never sets status, never approves.

---

0. Grounding — what exists and what we reuse

0.1 BACKEND (`~/PLACEDON/CODE/PLACEDON/BACKEND/`) — REUSE MAP

Existing module	State	Verdict for prototype
`app/deps.py`	JWT → RLS-scoped Supabase client	**Reuse as-is.** Every new `/v1` route depends on `get_auth`.
`app/jobs.py`, `app/v1_router.py`	Slice 1 jobs/Role-DNA/Reality-Card API, policy unit-tested	**Reuse.** Pattern-template for all new `/v1` modules (pure policy fns + thin router).
`app/websocket_router.py`	`WS /ws/{interview_id}`: dedupe by `message_id`, token streaming, superseded-connection handling	**Port** into new `WS /v1/interviews/{id}/live` (add auth + persistence). The reconnect/dedupe logic is good; keep it.
`app/live_runtime.py`	AoT orchestrator + layer2 (SBERT) + layer3 (bias guard) + layer5 loop; sufficiency stop (`should_end_interview`)	**Reuse the orchestration skeleton and the bias-guard (`SafeQuestionPipeline`) verbatim.** Replace its layer5 "fit score"/profile output with the evidence-unit pipeline (Pillar 3) — `fit.fit_score` is a universal-score smell and must not surface.
`app/session_manager.py` (Redis)	Session state store	**Reuse.** Add TTL + `schema_version` field.
`app/interaction_router.py` + `interaction_layer/voice/*`	Voice WS `/interaction/ws/{session_id}`, `build_stt()` factory (`MockSTT` / `WhisperSTT`), `MockTTS`, `AudioChunk` model (already supports base64), recovery/silence/turn managers	**This is the voice seam.** We add `DeepgramSTT` to the factory and a real TTS provider chain. The turn/recovery managers are directly reusable.
`app/tts_service.py` (`MacTTSService`, macOS `say`)	Dev-only hack	**Retire** behind a `TTSProvider` interface; keep as local-dev fallback only.
`llm/generator.py`, `llm/judge.py`, `llm/claude_axis.py`, `llm/openai_client.py`	Question generator + judge (GPT-4o, temp 0.1, JSON extract, deterministic calibration `_calibrate_output`)	**Reuse the shape**: LLM call → `extract_json` → Pydantic → deterministic calibration is exactly the right architecture.

Existing module	State	Verdict for prototype
		Extend with the extractor/fidelity-judge pair (Pillar 3). Cross-provider judging (Claude extracts, GPT-4o judges) is a feature — keep both clients. Remove the `print()` debug lines when touched.
`pipeline/` (`planner`, `context_builder`, `conversation_orchestrator`, `question_strategy`, `jd_parser`)	Adaptive planning	**Reuse.** `plan_next_step` + `generate_question` become the live interview brain; `context_builder` gets Role-DNA input.
`api_routes.py`, `analytics_router.py`	CSV ingest / rating / JSONL export	**Reuse as the eval/data pipeline** (Pillar 3.3) — not product API.

0.2 FRONTEND (`~/PLACEDON/PLACEDON-WEB/SRC/`) — CURRENT SEAMS

- `lib/api.ts` — `isLiveBackend()` gate (env-driven) + WS base URL. The **silent demo fallback lives here** (P0 fix).
- `lib/interview/useInterviewSession.ts` — WS hook against `/ws/{interview_id}`; extend, don't rewrite.
- `components/candidate/TrustPassport.tsx` — renders `report.overall` + `overallConfidence` (lines ~37–38, ~86–90). **P0: delete the universal score.**
- `components/candidate/{CandidateDashboard,CandidateMatches,CandidateApplications}.tsx` + `lib/mock/*` — mock surfaces to gate (P0) then progressively replace.
- `lib/supabase/{client,server,middleware}.ts` — auth wiring exists.

0.3 DB (SUPABASE `PLACEDON`, PROJECT `NFMTTCKZSBCXZHUSCZCK`)

Built: `profiles`, `demo_requests`, `companies`, `organization_members`, `jobs`, `role_dna_signals`, `job_reality_cards` (+ `is_org_member` SECURITY DEFINER; pending schema-move to `private`). Everything below adds to this with RLS-first migrations under `Code` repo root `supabase/migrations/` (learned the hard way: never a `supabase/` dir inside the backend package).

Pilot role family: **junior backend**. All prompts, eval cases, and Role-DNA templates target it first.

PILLAR 1 — Voice + Text interview (both real, both usable)

Text is the trunk; voice is a first-class input/output mode on the same trunk. One pipeline, two transports. Voice never creates a second assessment path.

1.a Architecture / data flow

Turn state machine (server-owned, per turn): `AI_SPEAKING → LISTENING → ENDPOINTING → PROCESSING → AI_SPEAKING` Candidate can type at any state (dual-mode composer); a typed submit short-circuits `LISTENING/ENDPOINTING`.

1.b Exact components to build

FRONTEND (NEXT.JS 16)

File	Purpose
`src/components/interview/ConsentGate.tsx`	Pre-interview consent: what's recorded, "assessed as text only", provider named, retention, switch-to-text-anytime. Blocks entry until accepted; writes `interview_consents`.
`src/components/interview/MicCheck.tsx`	getUserMedia permission flow, level meter, device picker, "sounds good?" playback-free check (level only — we never play their voice back for judgment).
`src/components/interview/InterviewRoom.tsx`	The room: question pane, live caption pane (partials), dual composer (text field + hold/toggle mic), calm status ("Saved" ack / "thinking…").
`src/components/interview/VoiceControls.tsx`	Mic toggle, mute, "switch to text" (always visible), barge-in ("tap to interrupt" while AI speaks).
`src/components/interview/TranscriptRail.tsx`	Running transcript of the session (their words, editable *before* submit in push-to-talk phase).
`src/lib/interview/useLiveInterview.ts`	Evolves `useInterviewSession.ts`: auth'd WS client for `/v1/interviews/{id}/live`, JSON + binary frames, reconnect with `last_message_id` replay, mode switching.
`src/lib/interview/useMicCapture.ts`	AudioWorklet capture → PCM16 16kHz mono, 100ms Float32→Int16 frames; client VAD level events; ring buffer (10s) to survive reconnects.
`src/lib/interview/useTtsPlayback.ts`	Streams `tts_chunk` into Web Audio (`AudioContext` + queue of `AudioBufferSourceNode`s); exposes `stop()` for barge-in.
`src/lib/interview/audio-worklet.ts`	The worklet processor (downsample + Int16 encode).

Why AudioWorklet PCM16 instead of MediaRecorder/webm-opus: deterministic across Chrome/Firefox/Safari (Safari's MediaRecorder container support is still inconsistent), no container parsing server-side, and Deepgram takes `encoding=linear16&sample_rate=16000` directly. Bandwidth is 32 KB/s — fine.

BACKEND (FASTAPI)

File	Purpose
`app/interviews.py`	Pure ops + schemas: create session, persist turns, consent records, mode switches (mirrors `jobs.py` pattern).
`app/interviews_router.py`	`POST /v1/interviews` (candidate creates; requires consent payload), `GET /v1/interviews/{id}`, `POST /v1/interviews/{id}/consent`, `WS /v1/interviews/{id}/live` (JWT in first message or query param — browsers can't set WS headers; validate before accept-loop).
`app/voice/__init__.py`, `app/voice/session.py`	Voice session orchestration: turn state machine, endpointing decisions, barge-in cancellation, mode-switch bookkeeping. Wraps the reusable `TurnManager` / `RecoveryManager` from `interaction_layer`.
`interaction_layer/voice/deepgram_stt.py`	`DeepgramSTT(BaseSTT)` — async streaming client (Deepgram Python SDK), emits partial/final `STTEvent`s; registered in `interaction_layer/voice/factory.py` (`stt_backend="deepgram"`).
`app/voice/tts_providers.py`	`TTSProvider` protocol + `ElevenLabsTTS`, `OpenAITTS`, `MacSayTTS` (dev). Streaming synth: sentence-chunked input → audio chunks out. `interaction_router`'s `MockTTS` and `tts_service.py` retire behind this.
`app/voice/policy.py`	Deterministic voice policy: what is stored (transcript yes, audio no), consent required per feature, provider allow-list.

WS message protocol (versioned, `"v":1`):

- client→server: `{"type":"answer_text","message_id","content"}` · binary audio frame (header byte + PCM) · `{"type":"audio_end"}` (push-to-talk) · `{"type":"barge_in"}` · `{"type":"mode_switch","mode":"text"|"voice"}` · `{"type":"clarify_request"}` (the two-way affordance from the audit B2)
- server→client: `{"type":"ack"}` (render "Saved" <150ms after final) · `{"type":"stt_partial"|"stt_final","transcript","confidence"}` · `{"type":"status","state":"thinking"}` · `{"type":"question_token"}` / `{"type":"question"}` · binary `tts_chunk` · `{"type":"tts_end"}` · `{"type":"session_complete"}` · `{"type":"error","code"}`

Reuse from `websocket_router.py`: `message_id` dedupe, superseded-connection close (1012), replay-last-question on reconnect. Reconnect contract: 3 failed WS retries → client offers "continue in text via HTTPS" (`POST /v1/interviews/{id}/turns` REST fallback, same pipeline).

SUPABASE TABLES (MIGRATION `0002_INTERVIEWS.SQL`)

```
interview_sessions(id, candidate_id → profiles, role_family, job_id nullable,
  status text check in ('created','in_progress','complete','abandoned'),
  mode_default text check in ('text','voice'), started_at, completed_at,
  runtime_snapshot jsonb)                          -- RLS: candidate owns row
interview_turns(id, session_id, turn_index, question_text, question_signal_id,
  answer_text_encrypted,                           -- pgsodium/pgcrypto; NEVER readable by
employers (no employer policy at all)
  answer_mode text check in ('text','voice'), stt_confidence numeric null,
  asked_at, answered_at)                           -- RLS: candidate-only
interview_consents(id, session_id, candidate_id, kind text check in
('interview','voice'),
  policy_version text, stt_provider text null, tts_provider text null,
  audio_retention text default 'none', consented_at)  -- RLS: candidate-only, append-
only
```

1.c Providers — comparison and recommendation

STT (the decision that matters):

	Deepgram Nova-3	AssemblyAI Universal-Streaming	faster-whisper (self-hosted GPU)
Streaming partials latency	~200–300ms	~300–500ms	1–3s realistically (chunked)
Endpointing/VAD events	Built-in (`speech_started`, endpoint config) — feeds barge-in + turn-taking directly	Basic end-of-turn detection	DIY (silero-vad)
Indian-English / Hinglish code-switching	Nova-3 multilingual handles en-hi code-switch; keyterm boosting	English-centric streaming	Whisper large-v3 decent but slow
Price	~~$0.0077/min streaming~~ ($0.46/hr)	~$0.15–0.47/hr	T4 spot ≈ $0.35/hr + ops + cold starts
Ops burden	zero	zero	GPU worker, autoscaling, model mgmt — a whole subsystem

Recommendation: Deepgram Nova-3 streaming. The built-in endpointing + speech-start events remove two hard client problems (turn-taking, barge-in), the Hinglish story matters for the India wedge, and at prototype volume (<2k interviews/mo) cost is noise. Keep `WhisperSTT` in the factory as the offline/batch re-transcription path (eval reprocessing) — that's where self-hosted whisper earns its keep, not live.

TTS: `TTSProvider` interface with two implementations from day one:

- **Default: ElevenLabs Flash v2.5** — ~75ms model TTFB, natural enough that voice mode *sells*; ~$0.06–0.11/min synthesized.
- **Cost fallback: OpenAI** `gpt-4o-mini-tts` — ~$0.015/min audio out, streaming, fine quality; env-flag swap (`TTS_PROVIDER`). Azure Neural is the enterprise/compliance alternative later; the interface makes it a one-file add.

Latency budget (end of candidate speech → AI audibly responding):

Segment	Budget
Endpoint detection (silence)	700ms (Deepgram endpoint config; also "I'm done" tap = 0ms)
Final transcript delivery	≤300ms
Ack render on client	≤150ms after final (perceived responsiveness — cheap, do it)
Planner + question generation first token	≤1,200ms (Claude Sonnet streaming; planner is one small call)
First sentence complete → TTS first chunk	≤400ms (synthesize sentence-by-sentence, never whole-answer)
Total p50 / p95	**≤2.5s / ≤4.0s**

If p95 breaches 4s, the fallback order is: cheaper/faster generator model for the *question* (extraction can lag async), then trim planner context. Never show dead air — the `status:thinking` state with the calm indicator is mandatory.

Cost per 20-min voice interview (candidate speaks ~12 min, AI ~5 min / ~4.5k chars): STT $0.15 + TTS $0.35 (ElevenLabs) or $0.08 (OpenAI) + LLM ~$0.30 (question gen + extract + judge, Sonnet-class) ≈ **$0.55–0.80**. Text-only interview ≈ **$0.30**. Both fine.

1.d Fairness + safety guardrails (voice-specific)

1. **Transcribe → assess as text only.** The assessment pipeline receives `answer_text` and nothing else. `stt_confidence` is used ONLY for "could you repeat that?" recovery (existing `RecoveryManager.on_low_confidence`), never as a feature in extraction, judging, matching, or ranking. Enforced structurally: the extractor/judge function signatures take `(question, answer_text)` — no audio metadata parameter exists.

2. **No prosody features, ever.** No WPM, filler-word counts, pause analysis, sentiment-from-audio. Code review checklist item; eval harness includes a "forbidden feature" grep test on the pipeline modules.
3. **Audio retention = none by default.** Frames stream through to STT and are dropped. `interview_consents.audio_retention` exists so a future explicit opt-in (e.g., candidate wants to re-listen) is a schema change away, not a policy default.
4. **Consent recorded with provider + policy version** before any frame leaves the browser. Mode switches logged in `interview_turns.answer_mode` so we can audit that voice/text candidates flow through identical assessment.
5. **Equity check in the eval harness:** paired eval cases (same answer content, one labeled voice-transcribed with realistic STT noise, one clean text) must produce band agreement ≥95%; if STT noise degrades bands, we add a transcript-cleanup pass, not a lower bar.
6. **Accessibility is the flip side:** voice helps candidates who type slowly; text helps candidates who can't/won't speak. Both always available mid-session — that's why the dual composer is non-negotiable.

1.e Phased build order

V0 — Live text interview (retire the mock). Wire `useLiveInterview` to `WS /v1/interviews/{id}/live` (auth'd port of `websocket_router.py`), persist `interview_sessions` / `interview_turns`, sufficiency stop, `clarify_request` affordance, ack + thinking states. *Accept:* a real signed-in candidate completes a junior-backend text interview on placedon.com; every turn is in Supabase under RLS; kill the network mid-answer and resume without losing the turn; questions reference the previous answer (spot-check 10 sessions); zero mock code paths executed.

V1 — Push-to-talk voice (1 week after V0). Mic capture, hold-to-talk, full-utterance PCM → Deepgram **prerecorded** endpoint → transcript shown for confirm/edit → submitted as text. No streaming, no TTS. This ships voice *input* with ~30% of the complexity. *Accept:* consent + mic-check flow complete; transcript editable before submit; `answer_mode='voice'` recorded; works on Chrome/Safari/Firefox at 320px width; declining mic lands you in text with zero friction.

V2 — Streaming STT + endpointing. Live partial captions, server endpointing, "I'm done" tap, REST/text fallback on reconnect failure. *Accept:* p50 partial-caption latency ≤500ms from speech; endpoint fires within 1s of silence; 3-retry → text fallback tested; barge-free conversation feels turn-based, not form-based (founder walkthrough sign-off).

V3 — TTS voice-out + barge-in (full duplex). Sentence-streamed TTS, playback queue, barge-in cancels server-side synth, `AI_SPEAKING` state visible. *Accept:* end-of-speech → AI audio p50 ≤2.5s / p95 ≤4s measured over 20 sessions; barge-in stops audio <300ms and the interrupted question is marked; voice+text transcripts of the same session are identical in the DB; cost per interview logged in `model_runs` and ≤$1.

1.f UX walkthrough

Ravi, voice interview on his phone. Opens the invite → ConsentGate explains: “You can speak or type. If you speak, we transcribe your words and assess *only the words* — never your accent or how fast you talk. Audio is not stored. Powered by Deepgram. Switch to typing anytime.” → mic check shows the level bar bouncing → first question renders as text *and* is spoken. He answers aloud; his words appear as live captions; 0.7s after he stops, “Saved ✓” appears, then a subtle “thinking…” shimmer, then: “You said you’d check the retry queue first — what would you look at if the queue was empty?” His actual phrase, spoken back in the follow-up. Mid-interview his train hits a tunnel; the socket drops; on reconnect the same question is re-presented with his last answer intact. In a quiet carriage he taps “switch to text” and finishes typing. The report card later shows quotes from both halves — indistinguishable, because they are.

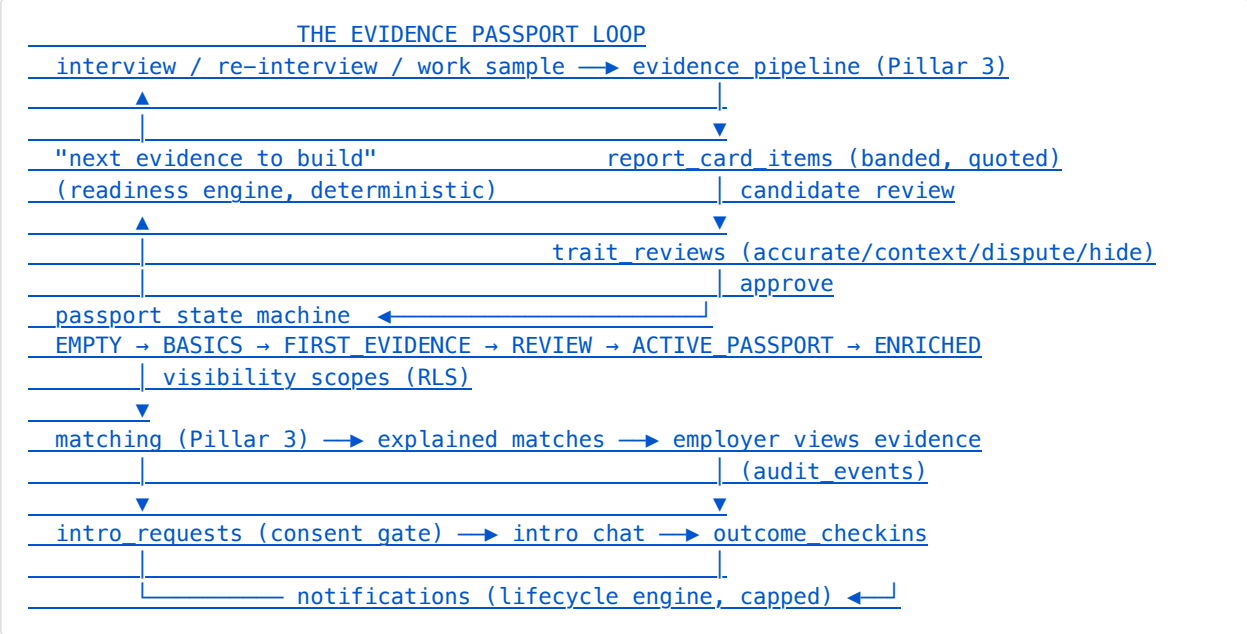
PILLAR 2 — Candidate profile + engagement (evidence, not social)

2.0 The tension, resolved

"Engagement" in consumer social means *time spent*; that metric is poison here — it selects for compulsion and popularity, both of which corrupt an evidence product. The retention asset we actually want is **accumulated, candidate-controlled evidence + trust**: every return visit should leave the candidate's passport *stronger* or their next step *clearer*. So we optimize **investment per session and honest progress**, not sessions per day. The candidate sticks because leaving means abandoning a compounding asset (their evidence graph and its standing consented distribution), not because we ping them.

Design frame (Hooked loop, deliberately de-fanged): **trigger → action → *earned* reward → investment**. All rewards are deterministic and true ("your debugging signal moved to *supported* because of this quote"), never variable-ratio dopamine.

2.a Architecture / data flow



Everything a candidate does feeds exactly one of: (1) evidence depth, (2) evidence accuracy (reviews), (3) distribution consent. There is no action in the product whose only output is a vanity counter.

2.b Exact components to build

FRONTEND

File	Purpose
`src/components/candidate/EvidencePassport.tsx`	Replaces `TrustPassport.tsx`. Per-role-family **signal grid**: each Role-DNA signal as a card with band (`supported` / `emerging` / `needs_more_evidence` / `not_yet_assessed`), the supporting quote(s), and provenance ("from your interview on Jul 18"). No numbers at the top.
`src/components/candidate/SignalCard.tsx`	One signal: band chip, quote, [Strengthen this signal] CTA (→ re-interview module or work-sample attach).
`src/components/candidate/ReviewQueue.tsx`	The post-interview review flow: per item — Accurate / Add context (inline text) / Dispute / Hide. Approval CTA gates passport activation.
`src/components/candidate/NextBestAction.tsx`	Single, prominent, state-machine-driven card on `/candidate` ("Review 2 new evidence items" / "One 15-min follow-up would strengthen your code-review signal" / "3 roles match — set your visibility"). Never more than ONE primary CTA.
`src/components/candidate/ReadinessGuide.tsx`	Honest gap view per target role family: which signals matter for junior-backend roles you're targeting, which are covered, concrete builders for each gap ("answer a focused follow-up" / "attach a repo where you reviewed code"). Never "you'll get hired if…".
`src/components/candidate/WorkSampleAttach.tsx`	Attach a repo link / doc / snippet to a *specific signal*; shows processing state → "summarized, awaiting your approval".

File	Purpose
`src/components/candidate/VisibilityControls.tsx`	Passport scopes: off / matched-roles-only / open-to-search; per-item overrides (hidden items never leave the candidate row — RLS-enforced).
`src/components/notifications/NotificationCenter.tsx` + `src/lib/notifications/useNotifications.ts`	In-app inbox; digest preferences; per-category toggles.
`src/components/intros/IntroThread.tsx` + `src/app/candidate/intros/`	The ONLY chat: thread view opened by an accepted intro; employer identity revealed only after acceptance.
`src/app/candidate/passport/`, `/candidate/review/`, `/candidate/readiness/`	Routes for the above; `/candidate` home becomes the NextBestAction state machine.

BACKEND

File	Purpose
`app/passport.py` + routes in a new `app/v1_candidate_router.py`	`GET /v1/me/passport`, `PATCH /v1/me/passport/visibility`, `GET/PUT /v1/me/preferences`, `GET /v1/me/readiness`. Readiness = deterministic diff(target role family Role-DNA template, current banded signals) — no LLM in the loop.
`app/reviews.py`	`POST /v1/reports/{id}/items/{item_id}/review` (accurate/context/dispute/hide), `POST /v1/reports/{id}/approve`. Dispute state-machine (open → reprocessed → resolved) — reprocessing re-runs extract+judge on the same turn text with the dispute note attached for the judge.
`app/artifacts.py`	`POST /v1/evidence/artifacts` (Supabase Storage upload URL + row), summarize job hook.
`app/intros.py`	`POST /v1/intros` (employer→candidate request; carries role + why), `POST /v1/intros/{id}/accept`
`app/notifications.py` + `workers/digest.py` (arq job)	Event fan-in → notification policy (category, caps, quiet hours) → in-app rows + email digest. Policy is a pure function → unit-tested table.
`app/events.py`	`product_events` writer (activation/return/depth metrics feed) + `audit_events` for every employer read of candidate evidence.

SUPABASE TABLES (MIGRATIONS `0003_PASSPORT.SQL`, `0004_INTROS_NOTIFICATIONS.SQL`)

```
candidate_preferences(candidate_id pk, role_families text[], locations, work_modes,
  salary_min_expectation, feedback_style, visibility text check in
('off','matched_only','searchable'))
report_cards(id, session_id, candidate_id, role_family, status check in
  ('building','ready_for_review','approved','superseded'), built_at, approved_at)
report_card_items(id, report_card_id, signal_id → role_dna_signals-or-taxonomy,
  claim text, band text check in ('supported','emerging','needs_more_evidence'),
  quote text, turn_id → interview_turns, judge_meta jsonb,
  candidate_state text check in
('unreviewed','accurate','context_added','disputed','hidden'),
  candidate_context text)
trait_reviews(id, item_id, candidate_id, action, note, created_at)   -- append-only
history (flywheel gold)
evidence_artifacts(id, candidate_id, signal_id, kind check in
('repo','document','snippet','link'),
  storage_path, summary text, status check in
('processing','ready','approved','rejected'))
intro_requests(id, job_id, company_id, candidate_id, message, status check in
  ('pending','accepted','declined','expired'), created_at, responded_at)
intro_messages(id, intro_id, sender_profile_id, body, created_at)
  -- RLS: participants only AND intro.status='accepted'
notifications(id, profile_id, category check in ('transactional','progress','guidance'),
  type, payload jsonb, read_at, created_at)
audit_events(id, actor_profile_id, action, object_type, object_id, meta jsonb,
created_at)
  -- INSERT-only for app; SELECT: candidates see events about THEIR objects (this powers
  -- "a real employer viewed your evidence" honestly — it's literally the audit log)
product_events(id, profile_id nullable, event, props jsonb, created_at)
```

RLS spine: candidates own their rows; employers can read `report_card_items` ONLY through a view joining visibility scope + `candidate_state NOT IN ('hidden','disputed')` + role-match scope. Raw `interview_turns` has **no employer policy at all**.

2.c The engagement loops (trigger → action → reward → investment)

#	Loop	Trigger	Action	Earned reward	Investment created
1	**First evidence**	Signup / role pick	12-min interview	Report card with real quotes about *you* (this is genuinely novel — most candidates have never seen structured evidence of their thinking)	Passport exists; review pending
2	**Review & control**	"Your report is ready" (transactional)	Review items, add context, approve	Visible control: "2 items hidden, passport approved — employers see exactly this"	Trust + labeled data (flywheel)
3	**Strengthen a signal**	ReadinessGuide gap or a match that says "emerging"	15-min focused re-interview on ONE signal, or attach work sample	Band upgrade with the new quote shown side-by-side with the old ("emerging → supported")	Deeper evidence; higher-quality matches
4	**Distribution**	New explained match (progress digest)	Set visibility / express interest	Match card explains *why* per dimension, incl. disclosed salary range	Consent graph grows
5	**Real-world signal**	Employer viewed evidence / intro request (transactional for intros; digested for views)	Accept/decline intro → chat	An actual human conversation — the only reward that matters in hiring	Outcome data at 30/90d (flywheel)
6	**Outcome check-in**	30/90-day post-intro nudge (guidance)	2-question check-in	Updated readiness guidance grounded in what actually happened	Re-ranker labels

Re-interview design (loop 3, the retention workhorse): a re-interview is a *module*, not a redo — 3–5 questions targeted at exactly one signal, created via `POST /v1/interviews` with `{"focus_signal_id": …}`; the planner (`pipeline/planner.py`) receives the focus and the prior evidence so it probes deeper, not wider. New items **supersede** on upgrade only — a worse follow-up performance never downgrades an approved band (candidate safety: practicing can't hurt you; the judge simply finds no *stronger* evidence). This asymmetry is what makes "try again" feel safe instead of punitive, and it's honest because bands assert evidence *found*, not absence.

Streaks — REFUSED, with the argument: streaks manufacture daily opens via loss-aversion. Hiring is episodic (weeks-scale); there is no honest daily action, so a streak would either be fake (open-the-app streaks) or push junk actions that pollute evidence quality and our own depth metrics. Worse, streaks systematically favor the time-rich over the busy — a bias with disparate-impact texture in a *hiring* product. What we build instead: **momentum framing** — "You've built evidence for 3 of 5 junior-backend signals" persists forever, decays never, and resumes exactly where you left off after a month away. Returning after absence is *welcomed* ("your passport is intact; one 15-min follow-up is still open"), never guilt-tripped ("you lost your streak").

Notification policy (the lifecycle engine), deterministic table:

Category	Examples	Delivery	Cap
Transactional	Intro request, intro message, report ready, dispute resolved	Immediate (push/email + in-app)	Uncapped (each is a real event the user must act on)
Progress	"Employers viewed your evidence (3 this week, junior-backend roles)" — aggregated from `audit_events`, identity NEVER revealed pre-intro; "new match"	Daily digest max, quiet hours 21:00–09:00 local	≤1/day
Guidance	Readiness updates, "a follow-up module is available"	Weekly digest	≤1/week

Employer-view notifications deserve the explicit call: the *event* is honest (it's the audit log) and it's the single most motivating true fact we can tell a candidate — but delivered real-time per-view it becomes a slot machine. Aggregated into the daily/weekly digest with role context and zero identity, it stays calm and true. Every category individually mutable; global mute one tap.

What we REFUSE to build (and why, so it stays refused):

- ❌ Likes/reactions/endorsements — popularity ≠ competence; imports bias.
- ❌ Followers/feeds — attention marketplace dynamics corrupt evidence incentives; moat is trust, not graph.
- ❌ Public profile view counters or "profile strength 87%" — vanity numerics; our completeness UI is the signal grid, which is information, not a score.
- ❌ Open DMs / recruiter cold-mail — the consent gate IS the product promise.
- ❌ Leaderboards / percentile-vs-others — inter-candidate comparison is a universal score wearing a costume.
- ❌ Daily streaks / login rewards — argued above.
- ❌ Gamified badges for volume ("10 interviews!") — volume isn't evidence. The only celebrated states are band transitions and real-world events, both of which are true.

2.d Fairness/safety guardrails

- Visibility default = `off` until explicit approval; disputes freeze the item's distribution instantly (RLS predicate on `candidate_state`).
- Every employer read audited (`audit_events`) and candidate-visible in aggregate — surveillance symmetry.
- Notification engine cannot be triggered by the LLM; only deterministic events fan in.
- Intro chat: rate limits per employer seat, report/block controls, retention policy; no attachment types that leak PII floods (v1: text-only).
- Copy review rule: guidance language always names *evidence*, never *the person* ("no code-review example observed yet", never "weak collaborator").

2.e Phased build order

E1 — Passport + review core (depends on V0 + Pillar 3 M-A). EvidencePassport, ReviewQueue, approve→visibility flow, `/candidate` home as state machine + NextBestAction. *Accept:* candidate completes interview → reviews → hides one item → approves; hidden item provably invisible to an employer JWT (RLS test); `TrustPassport.tsx` deleted; activation event instrumented.

E2 — Readiness + strengthen loop. ReadinessGuide, focused re-interview modules, band-upgrade-only supersede logic, WorkSampleAttach (repo link + doc first; summarization job). *Accept:* candidate with an "emerging" signal completes a follow-up module and sees `emerging → supported` with both quotes; a *worse* follow-up changes nothing; work sample attaches to a signal and lands in the review queue as a new item.

E3 — Notifications + audit surface. notification tables, policy function + unit table-tests, digests (arq), employer-view aggregation from `audit_events`. *Accept:* per-category caps verified in tests; quiet hours respected; candidate sees "viewed by 3 employers this week (junior-backend)" and no identities; opt-out rate instrumented.

E4 — Intros + the only chat (depends on matching M-C). intro_requests lifecycle, IntroThread, contact release on accept. *Accept:* full happy path employer→request→accept→chat; declined intro leaks zero identity either direction; messages API 403s on pending intros (test); report/block works.

Metrics (defined now, dashboarded from E1):

- Activation: % signups completing first interview ≤72h (target 40%); % of those approving a passport ≤7d (target 60%).
- Return: W1 and W4 return rate to a *meaningful* action (review/strengthen/visibility/intro — app-opens don't count).
- Evidence depth: median supported signals per active candidate @30d (target ≥3); % candidates with ≥1 re-interview or artifact @30d (target 25%).
- Trust proxies: dispute rate (healthy band 2–10%: 0% means nobody reads it, >10% means extraction quality problem), notification opt-out <10%, intro acceptance ≥50%.

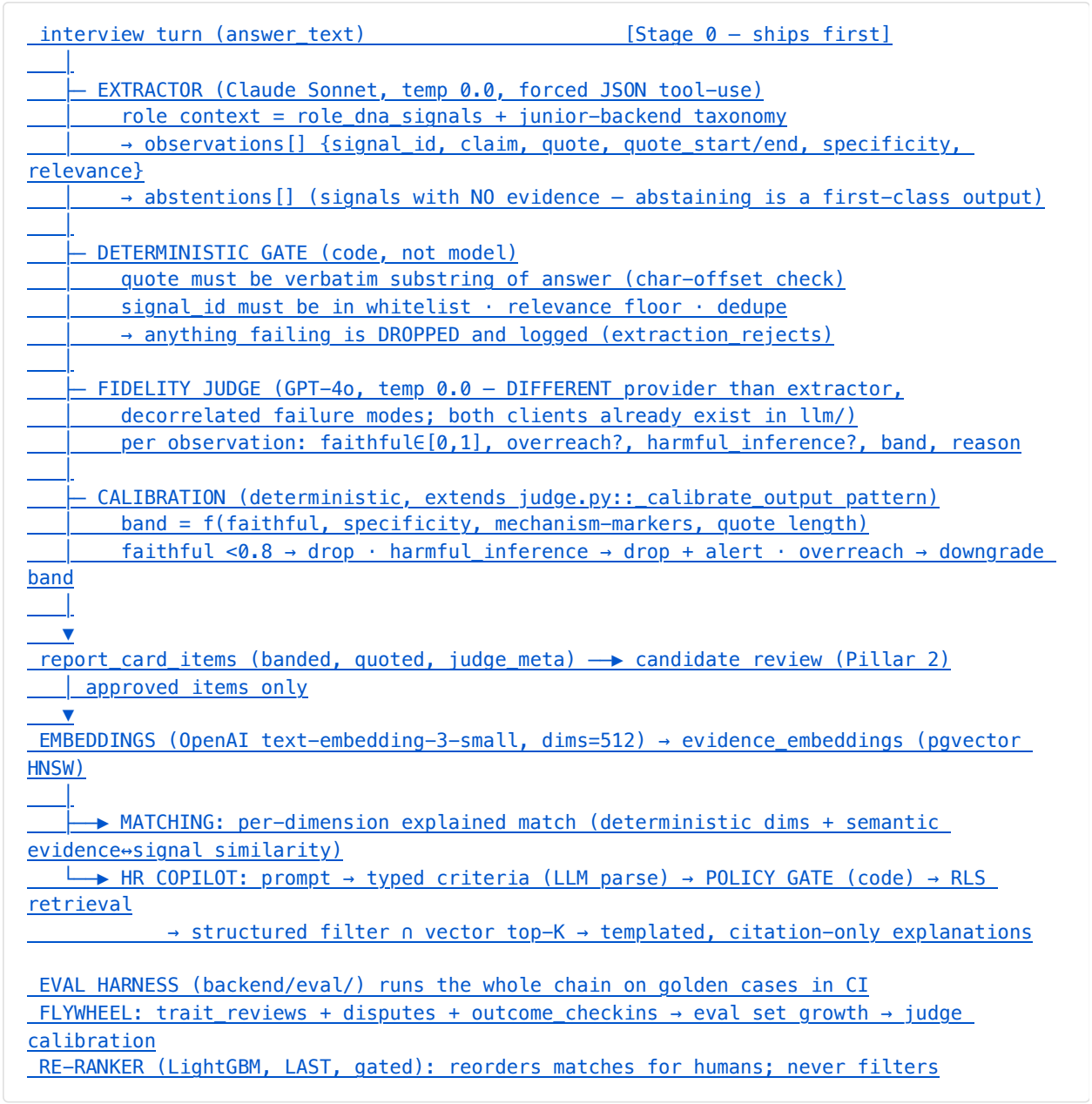
2.f UX walkthroughs

Aditi, week 1 → week 3. Tuesday: finishes her interview; that evening (transactional) — "Your evidence report is ready to review." She finds 6 items; 4 read true; one says *"Debugging: traces assumptions before proposing fixes — supported"* with her own sentence quoted, which she screenshots because no interviewer ever showed her that. One feels off — she taps **Dispute**; it vanishes from distribution instantly and reprocesses overnight. She approves the passport at `matched only`. Friday digest: "2 junior-backend matches. Zerodha-adjacent fintech, ₹9–12L disclosed, hybrid BLR — 3 of your 4 supported signals match; code-review evidence would strengthen this." She taps [Strengthen], books the 15-min module for Sunday, answers four questions about a real PR disagreement, and Monday sees *emerging → supported* with the new quote beside the old. Week 3, transactional: "An employer requested an intro for Backend Engineer (Payments)." She reads the role's Reality Card, accepts, and the chat opens — the first identity reveal in the whole story.

What Aditi never saw: a score, a streak, a follower count, a "complete your profile to rank higher" nag, or a recruiter she didn't consent to.

PILLAR 3 — AI/ML working prototype (end-to-end, buildable now)

3.a Architecture / data flow



3.b Exact modules / schemas / tables

(1) EXTRACT → JUDGE → BAND PIPELINE

Backend files:

- `llm/extractor.py` — new; mirrors `generator.py` structure. Claude via `claude_axis.py`, forced tool-use JSON.
- `llm/fidelity_judge.py` — new; reuses `openai_client.call_openai` + the `extract_json` → Pydantic → deterministic-calibration pattern proven in `judge.py`.
- `schemas/evidence_schema.py` — Pydantic:

```
class Observation(BaseModel):
    signal_id: str; claim: str            # ≤140 chars, present-tense, behavior not identity
    quote: str; quote_start: int; quote_end: int
    specificity: Literal["concrete","generic"]
    relevance: float                      # 0..1 vs the signal definition
class ExtractionOutput(BaseModel):
    observations: list[Observation]; abstentions: list[str]
class Verdict(BaseModel):
    observation_id: str; faithful: float; overreach: bool
    harmful_inference: bool               # protected class, health, family, accent, age…
    band: Literal["supported","emerging","needs_more_evidence"]; reason: str
```

- `pipeline/evidence_builder.py` — orchestrates extract→gate→judge→calibrate per turn; writes `report_card_items`; called async from the live session (question generation never waits on it).
- `app/reports.py` + routes: `GET /v1/reports/{id}`, `POST /v1/interviews/{id}/finalize` (assembles the report card when the session completes).
- `workers/` (arq on the existing Redis): `build_report_card`, `reprocess_dispute`, `embed_evidence`.

Prompt strategy: prompts live in `llm/prompts/{extractor,judge,copilot_parser}/vN.md`, versioned; every call logged to `model_runs(provider, model, prompt_version, prompt_hash, input_ref, latency_ms, cost_usd, output_jsonb)`. Extractor prompt hard-codes the abstention norm ("If the answer contains no evidence for a signal, list it in abstentions. Inventing evidence is the worst failure.") and the identity/behavior rule (claims describe *what was said/done*, never who the person is). Judge prompt receives ONLY (answer_text, observation) — not the candidate name, not the session history — so it physically cannot judge anything but fidelity.

How hallucinated claims die, in order: (1) verbatim-substring check kills fabricated quotes at zero LLM cost; (2) cross-provider judge scores whether the *claim* is entailed by the *quote* — `faithful < 0.8` drops it; (3) `harmful_inference=true` drops it AND raises an ops alert + eval case; (4) candidate review is the final human gate, and a dispute is a labeled failure that flows back to the eval set. Four independent layers; the report card only ever contains claims that survived all four.

(2) EMBEDDINGS + PGVECTOR

Model decision: OpenAI `text-embedding-3-small` with `dimensions=512` for the product corpus. Vs sentence-transformers MiniLM (already in layer2): MiniLM is fine for the runtime's internal turn-tracking and stays there untouched, but for HR-Copilot-grade semantic search, 3-small is markedly better on retrieval quality, needs no GPU/worker on the serving path, and costs $0.02/1M tokens — an entire candidate's evidence corpus embeds for <$0.001. 512 dims (native API truncation) keeps the HNSW index small and is within pgvector's comfortable range. Revisit self-hosting only if volume makes API cost visible (it won't for years).

Migration `0005_embeddings.sql`:

```
create extension if not exists vector;
create table evidence_embeddings (
  id uuid primary key default gen_random_uuid(),
  item_id uuid references report_card_items on delete cascade,
  candidate_id uuid not null, role_family text not null,
  kind text check (kind in ('claim','quote','artifact_summary')),
  content text not null, embedding vector(512) not null,
  created_at timestamptz default now());
create index evidence_embeddings_hnsw on evidence_embeddings
  using hnsw (embedding vector_cosine_ops) with (m = 16, ef_construction = 64);
-- RLS: candidate owns rows; employer access ONLY via the approved-evidence view
predicate
-- (embedding rows join back to report_card_items; same visibility predicate applies)
```

Embed on approval only (approved items are the retrieval corpus — un-approved evidence is never searchable). Row is deleted on hide/dispute (cascade + trigger).

Query flow (Copilot + matching):

```
employer prompt — LLM parse → typed CriteriaJSON {role_family, signals[],
constraints{location,mode,salary}, free_text}
  → POLICY GATE (app/copilot/policy.py — pure code):
      deny/strip protected + proxy criteria (age, gender, college-prestige, "young",
marital…)
      → refused with explanation, or rewritten criteria shown as chips (HR sees exactly
what ran)
  → SQL: structured filter (role_family, visibility scope, constraint flags) under the
EMPLOYER'S JWT (RLS)
  → pgvector: embed(free_text + signal defs) → top-K=50 cosine within the filtered set
(set ef_search=100)
  → rank: supported-signal count ▸ semantic sim ▸ preference alignment (deterministic,
inspectable weights)
  → EXPLAINER: templated output citing item_id quotes; "missing signals" listed; NO
free-generation about people —
      the LLM never writes the result text, a template does (nothing uncited can be
said)
```

Files: `app/copilot/{parser.py,policy.py,retrieval.py,explainer.py}` + `POST /v1/copilot/search`; sessions + results persisted (`hr_search_sessions`, `hr_search_results`) for audit and for the re-ranker's future training features. Policy gate is a pure function with a table-driven test suite of ~50 adversarial prompts from day one.

(3) EVALUATION HARNESS — THE CORE ASSET

`backend/eval/` (extends the existing `api_routes.py` JSONL export path):

- `eval/cases/junior_backend/*.jsonl` — golden cases, consented + de-identified:

```
{"case_id":"jb-041","question":"…","answer":"…",
 "permitted_observations":[{"signal_id":"debugging_method","claim_gist":"traces assumptions"}],
 "forbidden_observations":["accent","age","gaps","college_prestige"],
 "gold_bands":{"debugging_method":"supported"},
 "source":"synthetic|consented_deidentified","notes":"…"}
```

- `eval/run_eval.py` — runs extract→gate→judge on every case; writes `eval_runs` + `eval_case_results`.
- `eval/metrics.py` — **groundedness** (observations with verbatim quotes / total; ship-gate ≥0.98), **fidelity agreement** (band vs gold, quadratic-weighted kappa ≥0.75), **harmful-inference rate** (any forbidden observation extracted; ship-gate = 0 on the eval set), **abstention correctness** (does it abstain when gold has no evidence).
- `eval/copilot_cases.jsonl` — HR-prompt → expected policy outcome (allow/rewrite/refuse) pairs; gate = 100% on refuse-cases.
- Seeding: 60 synthetic junior-backend cases written/curated by us in week 1 (10 clean, 10 rambling, 10 keyword-stuffed, 10 with bait for forbidden inferences, 10 STT-noise-styled, 10 Hinglish-inflected), then grown by the flywheel. CI job: eval runs on every prompt-version bump; a regression blocks the prompt merge. Tables: `eval_cases`, `eval_runs(prompt_version, model, metrics jsonb)`, `eval_case_results`.

(4) DATA FLYWHEEL

Every trust interaction is a label: `trait_reviews` rows (accurate = positive label; dispute = extraction/judge failure; context-added = incompleteness signal) and `outcome_checkins(intro_id, actor, horizon check in ('30d','90d'), outcome check in ('advanced','offer','mutual_worth_it','not_useful'), note)` from both sides post-intro. Weekly arq job `export_eval_batch`: pulls disputed/context-added items where the candidate consented to research use (separate consent bit in `candidate_preferences`, default OFF), de-identifies (deterministic scrub of names/emails/orgs + human review queue before a case enters `eval/cases/`), and proposes eval candidates. Disputes double as judge-calibration data: Stage-3 calibration = tune the deterministic band thresholds (not model weights) to minimize disagreement with candidate reviews on held-out cases — measure, don't vibe.

(5) THE FAIRNESS-GATED LEARNED RE-RANKER — THE ONLY TRAINED MODEL

Deliberately last. Preconditions (all must hold): ≥300 intros with 30d outcomes; eval harness green for 4 consecutive weeks; audit demographic set exists (voluntary, stored in an isolated `audit_demographics` table, service-role-only, physically excluded from feature pipelines).

- **Model:** LightGBM `lambdarank`. Small-data-appropriate, explainable via SHAP, millisecond inference. No deep net — we won't have the data and we'd lose the explanation.
- **Features (~15, allow-listed in `ml/features.py`; anything not listed cannot enter):** supported-signal count/ratio vs the job's Role-DNA; per-signal band one-hots for the job's top-3 signals; mean/max cosine(evidence, signal definitions); preference↔Reality-Card alignment flags (mode, location, salary-in-range); evidence recency; artifact count; re-interview count. **Excluded forever:** anything demographic or proxy (college, name features, employment gaps, location-as-identity, session speed, answer length, voice-vs-text mode).
- **Labels:** graded from outcomes — 3 = offer/mutual-worth-it, 2 = advanced, 1 = intro accepted, 0 = none. Labels describe *the match*, never the person.
- **Gate before ANY exposure:** offline NDCG@10 must beat the transparent baseline by ≥5%; **adverse-impact ratio ≥0.80** (4/5ths rule) on top-10 exposure for every audit subgroup; subgroup calibration curves reviewed by a human. Then 2 weeks **shadow mode** (logged, not shown). Ships as a *re-ranker of the top-50 retrieved set only*, output = position, UI unchanged (dimensions still explain the match), env kill-switch `RERANKER_ENABLED`, monthly AIR re-audit written to `reranker_models(version, artifact_path, metrics jsonb, fairness jsonb, approved_by, activated_at)`. If fairness fails at any point → transparent ranking, no debate.

3.d Guardrails recap (pillar-wide)

LLMs extract/parse/judge under schemas; code gates everything (substring check, policy filter, band calibration, intro/message authz, search-ready policy). Copilot explanations are templates over citations. `harmful_inference` is a monitored production metric, not just an eval number. Model/prompt versions logged per run → any report card is fully reproducible.

3.e Phased build order

M-A — Evidence pipeline E2E (with V0; ~1.5 wk). extractor + gate + fidelity judge + calibration + `report_cards` assembly + `GET /v1/reports/{id}`; 60-case eval seeded; CI eval job. *Accept:* real interview → report card with banded, quoted items in <2 min of session end; eval: groundedness ≥0.98, harmful-inference 0, kappa ≥0.7 (initial), abstention works on the no-evidence cases; a fabricated-quote unit test proves the deterministic gate fires; cost/report logged (≤$0.15 target).

M-B — Embeddings + Copilot retrieval (~1.5 wk). pgvector migration, embed-on-approve worker, copilot parser+policy+retrieval+explainer, `/employer/copilot` UI (chips + cited results). *Accept:* the audit-doc example prompt returns cited, RLS-scoped results in <2s p95; all 50 adversarial policy prompts

refused/rewritten (100%); hidden/disputed items provably absent from results under an employer JWT; every search audited.

M-C — Explained matching (~1 wk). `matches` table + nightly + on-approve build job (dimension computation is deterministic; semantic sim from M-B), candidate `/candidate/matches` with the 4-dimension card incl. disclosed salary. *Accept:* candidate with approved passport sees ≥1 explained match against a search-ready job; every dimension traceable to rows (no bare numbers anywhere in the payload).

M-D — Flywheel + calibration (ongoing from M-A; jobs land ~wk 5). dispute-reprocess worker, outcome check-ins, weekly eval-batch export + review queue, threshold calibration round 1. *Accept:* a dispute reprocesses and resolves ≤24h; eval set >100 cases with ≥30 from real (consented, de-identified) sessions; calibration measurably reduces candidate-disagreement on held-out reviews.

M-E — Re-ranker (calendar-gated; realistically months out). Feature/label pipelines can be built early behind the gate; training waits for the 300-outcome threshold. *Accept:* the preconditions + gates in 3.b(5), verbatim. Shipping is a fairness-review decision, not an engineering one.

3.f UX walkthrough (pipeline made visible)

Aditi answers: *"Last month our webhook retries were duplicating payments. I assumed the queue was at fault, but I first checked the idempotency keys — turned out the key TTL was shorter than the retry window."* Extractor emits two observations (debugging_method: "verifies assumptions against system behavior before changing code", quote = her TTL sentence, offsets 118–207; incident_ownership: emerging) and abstains on code_review_collaboration. The substring check passes; the GPT-4o judge scores faithful 0.93/0.88, no overreach, no harmful inference; calibration bands them supported/emerging. Her report card shows both with her exact words. She disputes nothing, approves; two embedding rows appear. Next day an HR user types "junior backend who can debug production payment issues, hybrid Bengaluru" → parser → policy pass → RLS-filtered vector search surfaces her card: *"Debugging — supported: 'I first checked the idempotency keys…' · Missing: code-review collaboration → suggested human follow-up."* The HR user requests an intro; Aditi's phone shows the one notification that day that's actually worth getting.

UNIFIED SEQUENCING — one roadmap

Assumes 1 senior full-stack + Claude-assisted throughput; weeks are calendar, workstreams overlap where dependencies allow.

```
Wk 0   ── P0 TRUST FIXES (2–3 days, before anything)
Wk 1–3 ── M1: Real text interview + evidence pipeline   [V0 + M-A + eval seed]
Wk 3–5 ── M2: Passport + review + candidate home        [E1, finish Slice-1 employer UI]
Wk 5–7 ── M3: Semantic layer: Copilot + matching        [M-B + M-C]
Wk 6–8 ── M4: Engagement loops + notifications          [E2 + E3 + M-D jobs]
Wk 8–10 ─ M5: Intros + chat + outcome check-ins         [E4 + checkins]
Wk 9–12 ─ M6: Voice                                     [V1 → V2 → V3]
Gated  ── M7: Re-ranker                                 [M-E, data-gated]
```

P0 (Wk 0, ship independently):

1. `TrustPassport.tsx`: delete `overall` / `overallConfidence` rendering (lines ~37–38, 86–90) and the type fields; interim per-band summary until E1 replaces the component entirely.
2. `lib/api.ts` + `useInterviewSession.ts`: when `isLiveBackend()` is false, render a persistent "Scripted preview — not a live interview" banner and label the flow throughout; no silent impersonation.
3. Gate `CandidateDashboard/Matches/Applications` mock surfaces behind an explicit "Preview" chip + `NEXT_PUBLIC_PREVIEW_SURFACES` flag. *Demoable claim:* the live site no longer says anything untrue. (This also unblocks honest founder demos immediately.)

Milestone	Depends on	Demoable working prototype claim
M1 (wk 3)	P0; Redis; Anthropic+OpenAI keys	A stranger signs up on placedon.com, takes a real adaptive text interview, and a banded, quoted evidence report exists in Supabase under RLS. Eval harness green in CI.
M2 (wk 5)	M1	The full candidate promise: interview → review → hide/dispute → approve → controlled passport. Employer Role-DNA UI done (Slice 1 closes). **First honest end-to-end demo of the product thesis.**
M3 (wk 7)	M2 (approved evidence to search)	HR types plain English, gets policy-filtered, RLS-scoped, citation-backed candidates; candidates see explained matches with disclosed salary. **The two-sided demo.**
M4 (wk 8)	M2 (M3 enriches)	Candidates return for a reason: readiness gaps, strengthen loops, calm digests. Retention metrics live.
M5 (wk 10)	M3 + M4	Complete hiring loop: search → intro → consent → chat → outcome check-in. **The pilot-ready prototype** — run 10 real candidates + 2 friendly employers on it.
M6 (wk 12)	M1 (V1 can start wk 4 in parallel)	Voice interviews that actually work: speak answers, live captions, AI voice, barge-in, seamless text fallback — assessed identically to text. **The founder-wow demo.**
M7 (gated)	≥300 outcomes, 4 wk green evals, audit set	Learned re-ranking behind a passed fairness gate — or, equally acceptable, the documented decision not to ship it yet.

Opinionated calls, stated plainly:

- Voice V1 (push-to-talk) starts week 4 in parallel — it's cheap and de-risks the audio stack early — but full-duplex V3 stays behind the M5 pilot loop, because a working *hiring loop* beats a talking demo for every stakeholder except the demo itself.
- The eval harness is not a "quality" line item; it is the company's core AI asset and it starts in week 1 alongside M-A. Prompts merge only through it.
- The re-ranker is the only trained model and it can slip forever without hurting the prototype. Everything user-facing is real ML already (encoders + constrained LLMs + calibrated judges) — "we train a model" is a milestone the *data* earns, not the roadmap.
- Restore `PLACEDON_LIVE_INTERVIEW_HR_COPILOT_PLAN.md` into the repo (or commit this doc's §schemas as the replacement contract) before M1 merges, so table contracts have a single citable home.

Cross-cutting acceptance for calling it "a working prototype" (the founder's bar): every surface on placedon.com is either real or explicitly labeled preview; a candidate and an employer can complete the entire loop (job → interview → evidence → review → search → match → intro → chat) with zero mock data; voice input works; all AI outputs are quoted, banded, judged, and reproducible from `model_runs`; the eval harness gates prompt changes in CI; and no universal score exists anywhere in code, DB, or UI.